

# Chapter 1

## discuss the appropriate use of built-in data structures;

Computer programs manipulate data to solve real-world problems. The organization of data in computer memory directly dictates how efficiently algorithms can access, search, insert, and delete records. High-level programming languages provide **built-in data structures**, which are pre-packaged, highly optimized collection types supported natively by the language runtime and standard library.

Understanding the underlying mechanics of built-in data structures is vital for software performance and correctness. Selecting an inappropriate structure, such as using a dynamic array for frequent membership checks across millions of records, can transform a sub-second operation into an execution bottleneck lasting minutes. Knowing when to utilize each built-in structure enables engineers to write clean, performant code without reinventing common data management patterns.

This chapter examines the core built-in data structures found across modern high-level programming languages, including static arrays, dynamic arrays, tuples, dictionaries, and sets. It analyzes their physical memory layouts, operational time complexities, and behavioral trade-offs. Mastering these foundational structures provides the essential context required for object-oriented data structure design and formal algorithm analysis in subsequent chapters.

### 1.1 Memory Layout and Fundamental Characteristics

Built-in data structures organize data in physical system memory according to specific layout paradigms. These paradigms govern how memory is allocated, accessed, and modified during program execution.

#### 1.1.1 Contiguous versus Non-Contiguous Allocation

Memory allocation strategies are broadly divided into contiguous and non-contiguous arrangements. A **contiguous memory allocation** reserves a single, uninterrupted block of physical memory addresses for storing data elements. Because elements sit directly

adjacent to one another, the memory address of the  $i$ -th element can be calculated directly from the base memory address of the structure using constant time address arithmetic:

$$\text{Address}(i) = \text{Base Address} + (i \times \text{Element Size}) \quad (1.1)$$

In contrast, non-contiguous memory allocation distributes data elements across arbitrary memory locations linked together via memory pointers. While non-contiguous structures facilitate dynamic resizing without reallocating full blocks, they lose the ability to perform constant-time random access by integer index and incur additional pointer overhead.

### 1.1.2 Mutability and Structural Invariance

Data structures are classified by whether their contents or dimensions can change after initialization. A **mutable** structure allows modification, addition, or removal of elements in place without allocating a new structure in memory. Dynamic arrays and dictionaries are primary examples of mutable built-in structures.

An **immutable** structure cannot be modified once created. Any transformation applied to an immutable structure creates a completely new instance in memory. Immutability provides inherent thread safety and guarantees structural invariance, making immutable built-in structures like tuples ideal for read-only configurations, dictionary keys, and concurrent execution contexts.

Static Array Layout (Fixed Memory Block = 4 Elements):

|                 |                 |                 |                 |
|-----------------|-----------------|-----------------|-----------------|
| +-----+         | +-----+         | +-----+         | +-----+         |
| Address: 0x1000 | Address: 0x1004 | Address: 0x1008 | Address: 0x100C |
| Index: 0        | Index: 1        | Index: 2        | Index: 3        |
| Value: 42       | Value: 87       | Value: 15       | Value: 93       |
| +-----+         | +-----+         | +-----+         | +-----+         |

Dynamic Array Layout (Logical Size = 3, Allocated Capacity = 6):

|                |         |         |                        |                   |                   |
|----------------|---------|---------|------------------------|-------------------|-------------------|
| +-----+        | +-----+ | +-----+ | +-----+                | +-----+           | +-----+           |
| Ind: 0         | Ind: 1  | Ind: 2  | Unallocated Space      | Unallocated Space | Unallocated Space |
| Val: A         | Val: B  | Val: C  | (Reserved Buffer)      | (Reserved Buffer) | (Reserved Buffer) |
| +-----+        | +-----+ | +-----+ | +-----+                | +-----+           | +-----+           |
| ^ Base Pointer |         |         | ^ Next Insertion Point |                   |                   |

## 1.2 Linear Built-in Data Structures

Linear data structures maintain a sequential ordering among their elements, where each element has a distinct predecessor and successor (except for the boundary elements).

### 1.2.1 Static Arrays

A static **array** is a fixed-size, contiguous sequence of elements of a single uniform data type. The total capacity of a static array must be known at allocation time and cannot grow or

shrink during program execution.

Because all elements share identical byte sizes, static arrays support direct constant-time indexed access, denoted as  $O(1)$ . However, inserting or deleting elements at arbitrary positions requires shifting subsequent elements, resulting in linear time complexity,  $O(n)$ , where  $n$  is the number of elements.

### 1.2.2 Dynamic Arrays

A **dynamic array** provides contiguous sequential storage similar to a static array but automatically resizes itself when capacity is exhausted. Popular high-level language implementations include Python `list`, Java `ArrayList`, and C++ `std::vector`.

To minimize the overhead of frequent allocations, dynamic arrays utilize a growth factor strategy. When an element is added to a full array, the underlying execution environment performs the following sequence:

1. Allocates a new contiguous memory block, typically twice the size of the existing block (growth factor of 2).
2. Copies all existing elements from the old memory block to the new memory block.
3. Releases the old memory block back to the system allocator.
4. Inserts the new element into the next available slot.

Dynamic Array Growth and Reallocation Sequence:

Step 1: Array is Full (Capacity = 4, Size = 4)

```
+---+---+---+---+
| A | B | C | D | (Old Memory Block)
+---+---+---+---+
```

Step 2: Append 'E' triggers allocation of new expanded block (Capacity = 8)

```
+---+---+---+---+---+---+---+---+
|   |   |   |   |   |   |   |   | (New Memory Block)
+---+---+---+---+---+---+---+---+
```

Step 3: Copy existing elements and insert 'E'

```
+---+---+---+---+---+---+---+---+
| A | B | C | D | E |   |   |   | (Size = 5, Capacity = 8)
+---+---+---+---+---+---+---+---+
                ^ Next Insertion Point
```

Step 4: Release old memory block back to heap allocator

Although individual resizing operations require  $O(n)$  copy operations, amortized analysis proves that appending an element to a dynamic array executes in  $O(1)$  time on average over a sequence of appends.

### 1.2.3 Tuples

A **tuple** is an ordered, immutable collection of elements. Unlike standard static arrays, tuples can store heterogeneous data types across their slots while maintaining fixed positional semantics.

Because tuples are immutable, the runtime allocates exact memory blocks with zero dynamic overhead or growth buffers. This makes tuples memory-efficient and faster to instantiate than dynamic arrays. Tuples are predominantly used to group related data fields together (such as database rows or multi-value function returns) without creating custom class abstractions.

## 1.3 Associative and Set Structures

Associative and set structures organize data based on logical keys or content values rather than sequential integer positioning.

### 1.3.1 Dictionaries and Hash Maps

A **dictionary** (also called a hash map or associative array) is an unordered collection of key-value pairs, where each key must be unique and immutable. Dictionaries allow programs to retrieve values rapidly using their associated keys rather than scanning through an entire dataset sequentially.

Internally, dictionaries rely on a mathematical function called a **hash function**. The hash function accepts a key as input and computes an integer hash code, which maps directly to a specific slot (bucket) within an underlying array.

Dictionary Hash-Based Lookup Mechanism:

```
Key: "OSUN_ID_402"
  |
  v
[ Hash Function ]  ==> Computed Bucket Index: 3
  |
  v
Underlying Bucket Array:
+---+-----+
| 0 | Empty |
+---+-----+
| 1 | Key: "OSUN_ID_101" -> Value: "A+" |
+---+-----+
| 2 | Empty |
+---+-----+
| 3 | Key: "OSUN_ID_402" -> Value: "B" | <== Match Found (0(1) Access)
+---+-----+
```

When two distinct keys produce the same bucket index, a hash collision occurs. Dictionaries resolve collisions using collision handling strategies such as **chaining** (maintaining a linked list per bucket) or **open addressing** (probing subsequent array slots). When a high-quality hash function is used, search, insertion, and deletion operations execute in  $O(1)$  average time complexity.

### 1.3.2 Sets

A **set** is an unordered collection of unique elements. Sets mirror the mathematical properties of set theory, preventing duplicate values from existing within the same container.

Most language libraries implement sets using hash table backends (where elements serve as keys mapped to dummy values) or balanced search trees. Hash-based set implementations offer  $O(1)$  average time complexity for insertion, deletion, and membership testing. Sets also provide direct, built-in operations for union ( $\cup$ ), intersection ( $\cap$ ), and difference ( $\setminus$ ).

## 1.4 Comparative Performance Analysis and Selection Criteria

Choosing the correct built-in data structure requires balancing time complexity, space overhead, and programmatic constraints. Table 1.1 summarizes the asymptotic operational characteristics of fundamental built-in data structures.

To determine the optimal data structure for a given task, consider the following structural decision pipeline:

Structural Selection Decision Tree:

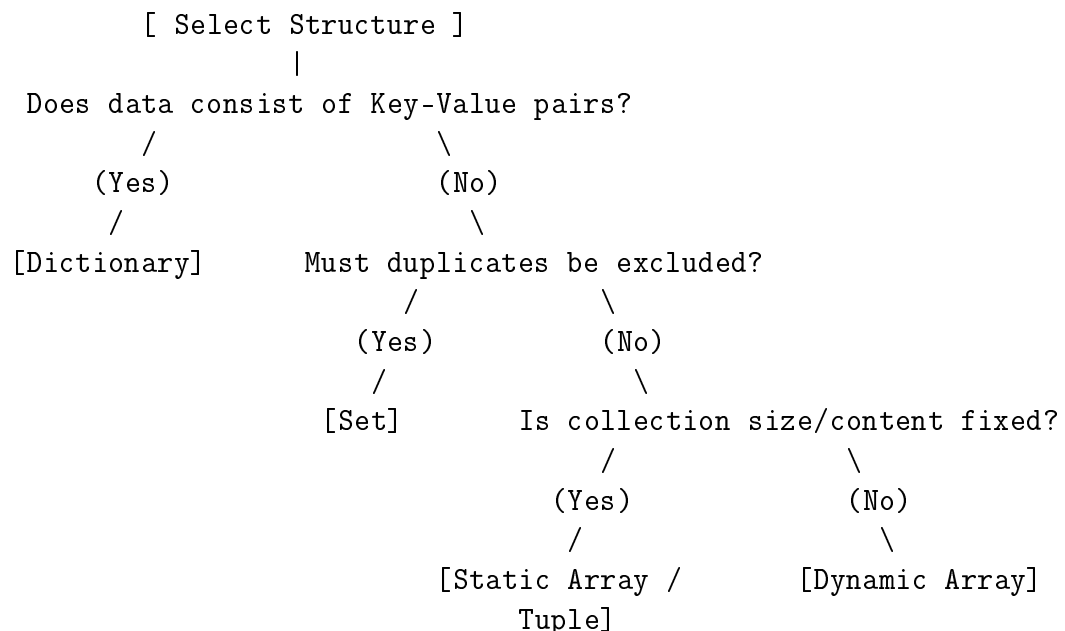


Table 1.1: Operational Complexity and Characteristics of Built-in Data Structures

| Data Structure | Access by Index | Search by Value | Insertion / Deletion         | Primary Use Case                                         |
|----------------|-----------------|-----------------|------------------------------|----------------------------------------------------------|
| Static Array   | $O(1)$          | $O(n)$          | $O(n)$                       | Fixed-size numeric or fixed-length sequences.            |
| Dynamic Array  | $O(1)$          | $O(n)$          | $O(1)$<br>Amortized (at end) | General-purpose ordered sequences with variable size.    |
| Tuple          | $O(1)$          | $O(n)$          | Not Supported (Immutable)    | Fixed multi-attribute records and composite lookup keys. |
| Dictionary     | Not Supported   | $O(1)$ Average  | $O(1)$ Average               | Rapid key-value lookups and relational mapping.          |
| Set            | Not Supported   | $O(1)$ Average  | $O(1)$ Average               | Uniqueness enforcement and fast membership checks.       |

1. **Identity versus Association:** If data items are indexed by natural sequential integers, select a dynamic array or static array. If data items are retrieved via unique identifiers (strings, UUIDs, compound keys), select a dictionary.
2. **Uniqueness Requirements:** If duplicate elements must be filtered out automatically or membership tests occur frequently, select a set.
3. **Mutability Constraints:** If the collection must remain fixed after creation to prevent accidental modification, select a tuple.
4. **Access Pattern Optimization:** If elements are accessed via arbitrary integer positions, prioritize linear contiguous structures to leverage hardware CPU cache lines.

## 1.5 Worked Examples

The following concrete engineering scenarios illustrate how to evaluate and select built-in data structures based on performance requirements.

### 1.5.1 Worked Example 1: Efficient Student Record Management

**Problem Statement:** A university registrar system receives continuous batches of student records. The system must perform two core tasks efficiently:

1. Retrieve student details instantly given a unique 8-digit Student Identification Number (Matric Number).
2. Maintain a log of incoming records in the exact order they arrive for auditing purposes.

**Analysis and Solution:** Using a dynamic array for Task 1 requires scanning elements sequentially, yielding an  $O(n)$  search time. For 100,000 students, scanning the array repeatedly creates severe performance penalties.

To achieve optimal performance, the system should combine two complementary built-in structures:

- **Dictionary for Fast Lookups:** Map each unique Matric Number (string key) to the corresponding student record object (value). This provides  $O(1)$  average time retrieval for Task 1.
- **Dynamic Array for Audit Logging:** Append incoming student record references sequentially to a dynamic array. This preserves insertion order and completes Task 2 in  $O(1)$  amortized time per insertion.

### 1.5.2 Worked Example 2: Filtering Duplicate IP Addresses

**Problem Statement:** A web server security daemon processes a log stream containing 10,000,000 IP address records. The log contains many repeating IP addresses. The daemon must compute the exact number of unique IP addresses present in the log and check whether specific suspicious IP addresses are present.

**Analysis and Solution:** If a dynamic array is used, checking if an IP address is already recorded requires searching through existing elements, taking  $O(k)$  time for an array of size  $k$ . Building the unique list across  $n$  log entries takes  $O(n^2)$  total operations:

$$\text{Total Operations} = \sum_{k=1}^n k = \frac{n(n+1)}{2} \approx 5 \times 10^{13} \text{ operations}$$

By choosing a **Set** structure instead:

1. Each IP address string is inserted into the set. The set hashes the string and performs insertion in  $O(1)$  average time. Duplicates are discarded automatically.
2. Total time complexity to process  $n$  entries drops to  $O(n)$ , executing roughly  $10^7$  operations instead of  $5 \times 10^{13}$ .
3. Subsequent queries checking whether a suspicious IP exists execute in  $O(1)$  average time.

## 1.6 Summary and Next Steps

Built-in data structures form the essential execution foundation of software applications. Arrays and dynamic arrays provide fast index-based positional access; tuples enforce immutability and record integrity; dictionaries enable constant-time key-based lookup; and sets optimize uniqueness checking and set-theoretic operations. Choosing the appropriate structure based on temporal and spatial complexity prevents severe performance bottlenecks.

While built-in data structures solve standard data management problems, complex domains often demand domain-specific data structures with customized behavior. The next chapter introduces object-oriented programming concepts, demonstrating how encapsulation, inheritance, and polymorphism enable engineers to build robust, custom data abstractions.

## Tutorial Questions

1. Explain the fundamental memory organization difference between contiguous and non-contiguous data structures. Identify how this difference impacts index-based element access time.
2. A software developer uses a dynamic array to store historical transaction records. Describe the internal steps taken by the runtime environment when an item is added to a dynamic array that has reached its maximum allocated capacity. What is the amortized time complexity of this operation?
3. Compare and contrast a tuple and a dynamic array across three distinct technical dimensions: memory allocation efficiency, mutability, and primary software usage cases.



4. An application performs frequent membership queries ("does item  $X$  exist in collection  $C$ ?") on a dataset containing 2,000,000 records. Evaluate the operational efficiency of performing this query using a dynamic array versus a set. Provide big- $O$  notation complexities for both approaches.
5. Explain how a dictionary achieves  $O(1)$  average time complexity for value retrieval. What condition causes this performance to degrade to  $O(n)$ , and how do modern hashing implementations mitigate this risk?
6. A system tracks live vehicle positions using geographical coordinate pairs (latitude, longitude). These coordinates must be updated continuously, but specific route history checkpoints must remain permanent and unalterable. Recommend the built-in data structure best suited for storing live coordinates versus permanent route checkpoints, justifying your selection for each.
7. Analyze the computational complexity trade-offs involved when using a static array versus a dynamic array in a memory-constrained embedded system with known, fixed bounds.